

Executive Summary

The celestial two-body pointing module computes a reference attitude that points the first spacecraft body-fixed axis $\hat{\mathbf{b}}_1$ towards a primary celestial object P while doing its best to point the second spacecraft body-fixed axis $\hat{\mathbf{b}}_2$ towards a secondary celestial object S . For example, the goal may be to point a sensor towards the center of a planet while keeping the solar panel normal pointed as close as possible at the Sun. The module outputs the reference MRP attitude $\boldsymbol{\sigma}_{\mathcal{R}/\mathcal{N}}$, the reference angular rate $\boldsymbol{\omega}_{\mathcal{R}/\mathcal{N}}$, and the reference angular acceleration $\dot{\boldsymbol{\omega}}_{\mathcal{R}/\mathcal{N}}$ of the reference frame $\mathcal{R}$ with respect to the inertial frame $\mathcal{N}$. The outputs are all expressed in inertial frame components.

Two pointing constraints in three-dimensional space is an overdetermined problem. Therefore, the primary pointing constraint is always met and is prioritized over the secondary constraint. The second constraint is met as closely as the remaining rotational degree of freedom allows.

This is the FP32 port of the Xmera `celestialTwoBodyPoint` module. Position and velocity inputs remain double-precision to preserve orbit-scale accuracy; the attitude / rate output is single-precision (float32).

Module Architecture

The module is split into two layers:

- The **adapter** (`CelestialTwoBodyPoint.h/.cpp`) is the SysModel-derived class that handles message I/O, validates configuration, builds an immutable `CelestialTwoBodyPointConfig` from public properties, and constructs the algorithm via two-phase initialization.
- The **algorithm** (`CelestialTwoBodyPointAlgorithm.h/.cpp`) is a pure C++23 class with no framework dependencies. It takes message payloads as input, computes the attitude reference, and returns a payload struct as output. It must not throw from `update()`.

A pure-C shim (`CelestialTwoBodyPoint.h/.cpp`) wraps the algorithm class for use by Ada/Adamant components via `extern "C"` bindings.

Adapter Layer

The adapter inherits from `SysModel`. It owns the input / output message hooks, validates that the required inputs are connected at `reset()` time, then constructs the algorithm via the two-phase init pattern. The secondary celestial body message is required.

Table 1: Module I/O Messages

Msg Variable Name	Msg Type	Description
<code>transNavInMsg</code>	<code>NavTransMsgF32Payload</code>	spacecraft inertial translational state (required)
<code>primaryCelBodyInMsg</code>	<code>EphemerisMsgF32Payload</code>	primary celestial body inertial state (required)
<code>secondaryCelBodyInMsg</code>	<code>EphemerisMsgF32Payload</code>	secondary celestial body inertial state (required)
<code>attRefOutMsg</code>	<code>AttRefMsgF32Payload</code>	reference attitude / rate / acceleration

Configuration

All tunable parameters are validated once in `CelestialTwoBodyPointConfig::create()` when the adapter's `reset()` runs; the algorithm never sees invalid parameter combinations.

Table 2: Configuration parameters

Parameter Name	Type	Valid range	Default	Description
<code>celestialBodyAlignmentThreshold</code>	<code>float</code>	$\geq 1e-6$ [rad]	1e-3	angle between the primary and secondary celestial bodies below which (or within the same margin of π) the secondary constraint is considered invalid and the constraint axis is fixed

Two-Phase Initialization

The Python usage follows the standard adapter lifecycle: set the public configuration properties, subscribe inputs, call `reset()` once, then drive `updateState()` each cycle. :

```

module = celestialTwoBodyPointF32.CelestialTwoBodyPoint()
module.celestialBodyAlignmentThreshold = 1.0 * macros.D2R
module.transNavInMsg.subscribeTo(nav_msg)
module.primaryCelBodyInMsg.subscribeTo(primary_cel_body_msg)
module.secondaryCelBodyInMsg.subscribeTo(secondary_cel_body_msg)

sim.AddModelToTask(task_name, module)
sim.InitializeSimulation()
sim.ExecuteSimulation()

```

If all input messages have not been connected when `reset()` runs, an `std::invalid_argument` is thrown. If `updateState()` is called before `reset()`, an `XmeraLifecycleException` is thrown.

Mathematical Formulation

Reference Frame Definition

Figure 1 shows the case where Mars is the primary celestial body and the Sun is the secondary one. Note that the origin of the desired reference frame $\mathcal{R}$ is the position of the spacecraft.

With knowledge of the spacecraft inertial position $\mathbf{r}_{B/N}$ and both primary and secondary celestial body inertial positions $\mathbf{r}_{P/N}$ and $\mathbf{r}_{S/N}$ (all of them relative to the inertial frame $\mathcal{N}$ and expressed in inertial frame components), the relative position of the celestial bodies with respect to the spacecraft is obtained by subtraction:

$$\begin{aligned}\mathbf{r}_{P/B} &= \mathbf{r}_{P/N} - \mathbf{r}_{B/N} \\ \mathbf{r}_{S/B} &= \mathbf{r}_{S/N} - \mathbf{r}_{B/N}\end{aligned}$$

Edge Case Guard 1

If the squared norm of either $\mathbf{r}_{P/B}$ or $\mathbf{r}_{S/B}$ are zero within a tolerance of `kMinNormSq`, the algorithm returns an identity reference attitude and zero reference rates. This is an impossible scenario where the spacecraft inertial position is identical to the inertial position of either celestial body.

The inertial derivatives of these position vectors are:

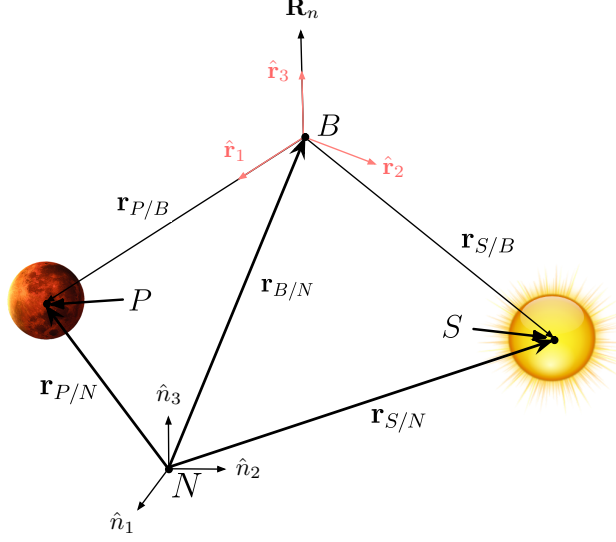


Figure 1: Illustration of the restricted two-body pointing reference frame $\mathcal{R} : \{\hat{\mathbf{r}}_1, \hat{\mathbf{r}}_2, \hat{\mathbf{r}}_3\}$ and the inertial frame $\mathcal{N} : \{\hat{\mathbf{n}}_1, \hat{\mathbf{n}}_2, \hat{\mathbf{n}}_3\}$.

$$\begin{aligned}
\dot{\mathbf{r}}_{P/B} &= \dot{\mathbf{r}}_{P/N} - \dot{\mathbf{r}}_{B/N} \\
\dot{\mathbf{r}}_{S/B} &= \dot{\mathbf{r}}_{S/N} - \dot{\mathbf{r}}_{B/N} \\
\ddot{\mathbf{r}}_{P/B} &= \ddot{\mathbf{r}}_{P/N} - \ddot{\mathbf{r}}_{B/N} \text{ (assumed zero)} \\
\ddot{\mathbf{r}}_{S/B} &= \ddot{\mathbf{r}}_{S/N} - \ddot{\mathbf{r}}_{B/N} \text{ (assumed zero)}
\end{aligned}$$

Note that while this documentation includes the full derivation including the accelerations of the planets with respect to the spacecraft, the module assumes that $\ddot{\mathbf{r}}_{P/B} = \ddot{\mathbf{r}}_{S/B} = 0$.

The normal vector $\mathbf{n}$ of the plane defined by $\mathbf{r}_{P/B}$ and $\mathbf{r}_{S/B}$ is computed through:

$$\mathbf{n} = \mathbf{r}_{P/B} \times \mathbf{r}_{S/B}$$

Edge Case Guard 2

If the angle between the celestial bodies $\mathbf{r}_{P/B}$ and $\mathbf{r}_{S/B}$ is, in absolute value, smaller than the configured `celestialBodyAlignmentThreshold` (or within the same margin of π), this is an edge case where the primary and secondary celestial bodies are aligned (or anti-aligned) as seen by the spacecraft. In this case, the primary pointing axis already satisfies both the primary and secondary pointing constraints. The secondary pointing axis is instead set in the direction of the primary body angular momentum vector:

$$\mathbf{r}_{S/B} = \mathbf{r}_{P/B} \times \dot{\mathbf{r}}_{P/B} \equiv \mathbf{h}_P$$

Edge Case Guard 3

This edge case falls within Edge Case 2. The relative position vector $\mathbf{r}_{P/B}$ and the relative velocity vector $\dot{\mathbf{r}}_{P/B}$ must not be collinear or the Edge Case 2 fallback constraint axis $\mathbf{r}_{P/B} \times \dot{\mathbf{r}}_{P/B}$ will be zero and undefined. In this case, the algorithm returns an identity reference attitude and zero reference rates. The angle between these vectors is computed and checked to ensure it is greater than a `kSmallAngle` threshold.

Setting the secondary constraint axis in the direction of the primary body angular momentum vector $\mathbf{h}_P$ ensures the axis will always be valid. ($\mathbf{r}_{P/B}$ and $\mathbf{r}_{S/B}$ are now perpendicular). The first and second time derivatives are:

$$\dot{\mathbf{r}}_{S/B} = \mathbf{r}_{P/B} \times \dot{\mathbf{r}}_{S/B}$$

$$\ddot{\mathbf{r}}_{S/B} = \dot{\mathbf{r}}_{P/B} \times \ddot{\mathbf{r}}_{S/B} \text{ (assumed zero)}$$

The inertial time derivative of $\mathbf{n}$ is found using the chain differentiation rule:

$$\dot{\mathbf{n}} = \dot{\mathbf{r}}_{P/B} \times \mathbf{r}_{S/B} + \mathbf{r}_{P/B} \times \dot{\mathbf{r}}_{S/B}$$

The second time derivative of $\mathbf{n}$ is found similarly:

$$\ddot{\mathbf{n}} = \ddot{\mathbf{r}}_{P/B} \times \mathbf{r}_{S/B} + \mathbf{r}_{P/B} \times \ddot{\mathbf{r}}_{S/B} + 2\dot{\mathbf{r}}_{P/B} \times \dot{\mathbf{r}}_{S/B}$$

Assuming the celestial body accelerations relative to the spacecraft are zero, the above expression simplifies to:

$$\ddot{\mathbf{n}} = 2\dot{\mathbf{r}}_{P/B} \times \dot{\mathbf{r}}_{S/B}$$

As illustrated in Figure 1, the base vectors of the desired reference frame $\mathcal{R}$ are defined as following:

$$\begin{aligned} {}^{\mathcal{N}}\hat{\mathbf{r}}_1 &= \frac{\mathbf{r}_{P/B}}{|\mathbf{r}_{P/B}|} \\ {}^{\mathcal{N}}\hat{\mathbf{r}}_3 &= \frac{\mathbf{n}}{|\mathbf{n}|} \\ {}^{\mathcal{N}}\hat{\mathbf{r}}_2 &= \hat{\mathbf{r}}_3 \times \hat{\mathbf{r}}_1 \end{aligned}$$

Since the position vectors are given in terms of inertial $\mathcal{N}$ -frame components, the DCM from the inertial frame $\mathcal{N}$ to the desired reference frame $\mathcal{R}$ is:

$$[\mathcal{R}\mathcal{N}] = [\mathcal{N}\mathcal{R}]^T = [{}^{\mathcal{N}}\hat{\mathbf{r}}_1, \quad {}^{\mathcal{N}}\hat{\mathbf{r}}_2, \quad {}^{\mathcal{N}}\hat{\mathbf{r}}_3]^T$$

The DCM result is converted to an MRP $\sigma_{\mathcal{R}/\mathcal{N}}$ and returned.

Reference Frame Basis Vector Time Derivatives

The first and second time derivatives of the base vectors that compound the reference frame $\mathcal{R}$ are needed in the following sections to compute the reference angular velocity and acceleration. Several lines of algebra lead to the following sets:

$$\begin{aligned} {}^{\mathcal{N}}\dot{\hat{\mathbf{r}}}_1 &= ([I_{3 \times 3}] - \hat{\mathbf{r}}_1 \hat{\mathbf{r}}_1^T) \frac{\dot{\mathbf{r}}_{P/B}}{|\mathbf{r}_{P/B}|} \\ {}^{\mathcal{N}}\dot{\hat{\mathbf{r}}}_3 &= ([I_{3 \times 3}] - \hat{\mathbf{r}}_3 \hat{\mathbf{r}}_3^T) \frac{\dot{\mathbf{n}}}{|\mathbf{n}|} \\ {}^{\mathcal{N}}\dot{\hat{\mathbf{r}}}_2 &= \dot{\hat{\mathbf{r}}}_3 \times \hat{\mathbf{r}}_1 + \hat{\mathbf{r}}_3 \times \dot{\hat{\mathbf{r}}}_1 \\ {}^{\mathcal{N}}\ddot{\hat{\mathbf{r}}}_1 &= \frac{1}{|\mathbf{r}_{P/B}|} (([I_{3 \times 3}] - \hat{\mathbf{r}}_1 \hat{\mathbf{r}}_1^T) \ddot{\mathbf{r}}_{P/B} - 2\dot{\hat{\mathbf{r}}}_1(\hat{\mathbf{r}}_1 \cdot \dot{\mathbf{r}}_{P/B}) - \hat{\mathbf{r}}_1(\dot{\hat{\mathbf{r}}}_1 \cdot \dot{\mathbf{r}}_{P/B})) \\ {}^{\mathcal{N}}\ddot{\hat{\mathbf{r}}}_3 &= \frac{1}{|\mathbf{n}|} (([I_{3 \times 3}] - \hat{\mathbf{r}}_3 \hat{\mathbf{r}}_3^T) \ddot{\mathbf{n}} - 2\dot{\hat{\mathbf{r}}}_3(\hat{\mathbf{r}}_3 \cdot \dot{\mathbf{n}}) - \hat{\mathbf{r}}_3(\dot{\hat{\mathbf{r}}}_3 \cdot \dot{\mathbf{n}})) \\ {}^{\mathcal{N}}\ddot{\hat{\mathbf{r}}}_2 &= \ddot{\hat{\mathbf{r}}}_3 \times \hat{\mathbf{r}}_1 + \hat{\mathbf{r}}_3 \times \ddot{\hat{\mathbf{r}}}_1 + 2\dot{\hat{\mathbf{r}}}_3 \times \dot{\hat{\mathbf{r}}}_1 \end{aligned}$$

Again, assuming the celestial body accelerations relative to the spacecraft are zero, $\ddot{\mathbf{r}}_1$ simplifies to

$${}^{\mathcal{N}}\ddot{\mathbf{r}}_1 = -2\dot{\mathbf{r}}_1(\dot{\mathbf{r}}_1 \cdot \dot{\mathbf{r}}_{P/B}) - \dot{\mathbf{r}}_1(\dot{\mathbf{r}}_1 \cdot \dot{\mathbf{r}}_{P/B})$$

Angular Velocity and Acceleration Descriptions

Using the identity $\mathbf{a} \cdot (\mathbf{b} \times \mathbf{c}) = \mathbf{b} \cdot (\mathbf{c} \times \mathbf{a})$, the following elegant expressions are found:

$$\begin{aligned}\hat{\mathbf{r}}_3 \cdot \dot{\hat{\mathbf{r}}}_2 &= \hat{\mathbf{r}}_3 \cdot (\omega_{\mathcal{R}/\mathcal{N}} \times \hat{\mathbf{r}}_2) = \omega_{\mathcal{R}/\mathcal{N}} \cdot \hat{\mathbf{r}}_1 \\ \hat{\mathbf{r}}_1 \cdot \dot{\hat{\mathbf{r}}}_3 &= \hat{\mathbf{r}}_1 \cdot (\omega_{\mathcal{R}/\mathcal{N}} \times \hat{\mathbf{r}}_3) = \omega_{\mathcal{R}/\mathcal{N}} \cdot \hat{\mathbf{r}}_2 \\ \hat{\mathbf{r}}_2 \cdot \dot{\hat{\mathbf{r}}}_1 &= \hat{\mathbf{r}}_2 \cdot (\omega_{\mathcal{R}/\mathcal{N}} \times \hat{\mathbf{r}}_1) = \omega_{\mathcal{R}/\mathcal{N}} \cdot \hat{\mathbf{r}}_3\end{aligned}$$

The time derivative of the above results can be easily taken to obtain:

$$\begin{aligned}\dot{\omega}_{\mathcal{R}/\mathcal{N}} \cdot \hat{\mathbf{r}}_1 &= \dot{\hat{\mathbf{r}}}_3 \cdot \dot{\hat{\mathbf{r}}}_2 + \hat{\mathbf{r}}_3 \cdot \ddot{\hat{\mathbf{r}}}_2 - \omega_{\mathcal{R}/\mathcal{N}} \cdot \dot{\hat{\mathbf{r}}}_1 \\ \dot{\omega}_{\mathcal{R}/\mathcal{N}} \cdot \hat{\mathbf{r}}_2 &= \dot{\hat{\mathbf{r}}}_1 \cdot \dot{\hat{\mathbf{r}}}_3 + \hat{\mathbf{r}}_1 \cdot \ddot{\hat{\mathbf{r}}}_3 - \omega_{\mathcal{R}/\mathcal{N}} \cdot \dot{\hat{\mathbf{r}}}_2 \\ \dot{\omega}_{\mathcal{R}/\mathcal{N}} \cdot \hat{\mathbf{r}}_3 &= \dot{\hat{\mathbf{r}}}_2 \cdot \dot{\hat{\mathbf{r}}}_1 + \hat{\mathbf{r}}_2 \cdot \ddot{\hat{\mathbf{r}}}_1 - \omega_{\mathcal{R}/\mathcal{N}} \cdot \dot{\hat{\mathbf{r}}}_3\end{aligned}$$

Note that $\omega_{\mathcal{R}/\mathcal{N}} \cdot \hat{\mathbf{r}}_1$ is the first component of the reference angular velocity $\omega_{\mathcal{R}/\mathcal{N}}$. The reference angular velocity is therefore:

$${}^{\mathcal{R}}\omega_{\mathcal{R}/\mathcal{N}} = \begin{bmatrix} \omega_{\mathcal{R}/\mathcal{N}} \cdot \hat{\mathbf{r}}_1 \\ \omega_{\mathcal{R}/\mathcal{N}} \cdot \hat{\mathbf{r}}_2 \\ \omega_{\mathcal{R}/\mathcal{N}} \cdot \hat{\mathbf{r}}_3 \end{bmatrix} = \begin{bmatrix} \hat{\mathbf{r}}_3 \cdot \dot{\hat{\mathbf{r}}}_2 \\ \hat{\mathbf{r}}_1 \cdot \dot{\hat{\mathbf{r}}}_3 \\ \hat{\mathbf{r}}_2 \cdot \dot{\hat{\mathbf{r}}}_1 \end{bmatrix}$$

Similarly for the angular acceleration:

$${}^{\mathcal{R}}\dot{\omega}_{\mathcal{R}/\mathcal{N}} = \begin{bmatrix} \dot{\omega}_{\mathcal{R}/\mathcal{N}} \cdot \hat{\mathbf{r}}_1 \\ \dot{\omega}_{\mathcal{R}/\mathcal{N}} \cdot \hat{\mathbf{r}}_2 \\ \dot{\omega}_{\mathcal{R}/\mathcal{N}} \cdot \hat{\mathbf{r}}_3 \end{bmatrix} = \begin{bmatrix} \dot{\hat{\mathbf{r}}}_3 \cdot \dot{\hat{\mathbf{r}}}_2 + \hat{\mathbf{r}}_3 \cdot \ddot{\hat{\mathbf{r}}}_2 - \omega_{\mathcal{R}/\mathcal{N}} \cdot \dot{\hat{\mathbf{r}}}_1 \\ \dot{\hat{\mathbf{r}}}_1 \cdot \dot{\hat{\mathbf{r}}}_3 + \hat{\mathbf{r}}_1 \cdot \ddot{\hat{\mathbf{r}}}_3 - \omega_{\mathcal{R}/\mathcal{N}} \cdot \dot{\hat{\mathbf{r}}}_2 \\ \dot{\hat{\mathbf{r}}}_2 \cdot \dot{\hat{\mathbf{r}}}_1 + \hat{\mathbf{r}}_2 \cdot \ddot{\hat{\mathbf{r}}}_1 - \omega_{\mathcal{R}/\mathcal{N}} \cdot \dot{\hat{\mathbf{r}}}_3 \end{bmatrix}$$

Alternatively, the rates can be computed in a compact manner using the elegant kinematic equations:

$$\begin{aligned}{}^{\mathcal{R}}\tilde{\omega}_{\mathcal{R}/\mathcal{N}} &= -[\dot{C}][C]^T \\ {}^{\mathcal{R}}\tilde{\dot{\omega}}_{\mathcal{R}/\mathcal{N}} &= -[\ddot{C}][C]^T - [\dot{C}][\dot{C}]^T\end{aligned}$$

where $[C] = [\mathcal{R}\mathcal{N}]$.

The rates can be expressed in the inertial frame using the $[\mathcal{N}\mathcal{R}]$ DCM.:

$$\begin{aligned}{}^{\mathcal{N}}\omega_{\mathcal{R}/\mathcal{N}} &= [\mathcal{N}\mathcal{R}] {}^{\mathcal{R}}\omega_{\mathcal{R}/\mathcal{N}} \\ {}^{\mathcal{N}}\dot{\omega}_{\mathcal{R}/\mathcal{N}} &= [\mathcal{N}\mathcal{R}] {}^{\mathcal{R}}\dot{\omega}_{\mathcal{R}/\mathcal{N}}\end{aligned}$$

Assumptions and Limitations

- Requirement: Both the primary and secondary celestial bodies are required be configured.
- Assumption: The acceleration of the spacecraft with respect to the planets is assumed to be negligible ($\ddot{\mathbf{r}}_{P/B} = \ddot{\mathbf{r}}_{S/B} = 0$).

- Edge Case 1: The spacecraft must not be coincident with either celestial body, otherwise the corresponding pointing direction is undefined. In this edge case, identity reference attitude and zero reference rates are returned.
- Edge Case 2: If the celestial bodies are aligned within the configured `celestialBodyAlignmentThreshold`, this is an edge case where the primary and secondary celestial bodies are aligned (or anti-aligned) as seen by the spacecraft. In this case, the primary pointing axis already satisfies both the primary and the secondary constraints and a new constraint axis is defined.
- Edge Case 3: (within Edge Case 2) The relative position vector $\mathbf{r}_{P/B}$ and the relative velocity vector $\dot{\mathbf{r}}_{P/B}$ must not be collinear or the Edge Case 2 fallback constraint axis $\mathbf{r}_{P/B} \times \dot{\mathbf{r}}_{P/B}$ will be zero and undefined.
- Position and velocity inputs are read in double precision to preserve orbit-scale accuracy. The attitude / rate output is single-precision; expect a relative accuracy of $\sim 10^{-6}$ on $\boldsymbol{\sigma}_{\mathcal{R}/\mathcal{N}}$ and $\boldsymbol{\omega}_{\mathcal{R}/\mathcal{N}}$.